

Lesson: Handling Routes and Middleware

Introduction

In this lesson, we will explore the core concepts of routes and middleware in Express.js, a fundamental part of building scalable and maintainable web applications. You will learn how to implement CRUD operations, organize routes using Express Router, and leverage middleware for tasks like logging, authentication, and request processing. By the end of this lesson, you will be able to structure your backend APIs efficiently and secure them using JWT-based authentication.

Learning Outcomes

By the end of this lesson, you will be able to:

1. Understand how to implement Create, Read, Update, and Delete (CRUD) functionalities using HTTP methods like GET, POST, PUT, and DELETE.
 2. Learn how to modularize your code by separating route logic into dedicated files, making your application easier to scale and maintain.
 3. Gain hands-on experience creating custom middleware for tasks like request logging, authentication, and more.
 4. Develop secure user authentication mechanisms using JSON Web Tokens (JWT), ensuring that only authorized users can access specific resources.
 5. Use tools like Postman to test and validate your API's functionality, ensuring that each endpoint behaves as expected.
-

Routing

Routing is the mechanism by which a web application determines **how to respond to client requests** based on the requested URL path and HTTP method. Whether the request is for retrieving a homepage, submitting form data, or modifying a resource, routing connects the **incoming request** to the appropriate **handler function** that produces a response.

At its core, a route is defined by three components:

1. **Endpoint (Path):** A specific URL or URL pattern (e.g., `/home`, `/users/:id`)
2. **HTTP Method:** The action requested by the client (e.g., `GET`, `POST`, `PUT`, `DELETE`)
3. **Handler Function:** The logic that runs when the request matches both the method and the path

Together, these components enable the application to respond accurately to different types of interactions.

The Role of Routing in Application Logic

Routing plays a central role in determining how a web application interprets and responds to various client requests. It acts as the connective layer between the **structure of incoming HTTP requests** and the **server-side logic** that handles them.

Every action—visiting a page, submitting a form, or requesting data—relies on a well-defined route to direct the request appropriately. Without structured routing, applications become difficult to maintain, extend, and secure.

Clear routing enables:

- A clean separation of concerns between URLs and logic
- Predictable behavior for both APIs and user interfaces
- Easier maintenance, refactoring, and scalability
- Controlled access to exposed functionality

By organizing routes carefully, developers ensure that the application remains consistent and modular as it grows in complexity.

Types of Routes

Modern applications use several types of routes to structure interactions effectively:

- **Static Routes**

Respond to fixed paths (e.g., `/about`, `/login`) where the response does not depend on parameters.

- **Dynamic Routes**

Include route parameters (e.g., `/users/:id`) to respond to variable input such as user IDs, product SKUs, or post slugs.

- **Nested Routes**

Reflect resource hierarchies (e.g., `/users/:userId/posts/:postId`) and are commonly used in RESTful APIs.

- **Wildcard Routes**

Match patterns or undefined paths, useful for 404 pages or fallback logic.

Routing in Action

When a client sends a request, the server performs the following:

1. Matches the request path to a defined route.
2. Confirms that the HTTP method matches the route definition.
3. Invokes the corresponding handler function with any route parameters.

Here's a simple example in pseudocode:

```
app.get('/users/:id', function (req, res) {
  const userId = req.params.id;
  // Logic to retrieve user by ID
  res.send(user);
});
```

This route handles any **GET** request to `/users/:id`, where `:id` is dynamically extracted and passed to the handler. With this understanding of how routing connects HTTP requests to application logic, we're ready to define and implement routes for the core operations of any application—creating, reading, updating, and deleting resources.

Creating GET, POST, PUT, DELETE Routes

A foundational aspect of web application development is defining how the server responds to different HTTP methods. These methods—**GET**, **POST**, **PUT**, and **DELETE**—are designed to reflect standard **CRUD** operations: Create, Read, Update, and Delete. Routing these methods to the appropriate logic enables your application to interact with client requests in a clear and predictable way.

Each route in your application should combine three components:

1. **HTTP Method** – The type of action requested by the client.
2. **Endpoint** – The URL path that identifies the target resource.
3. **Handler Function** – The logic executed when a request matches the method and endpoint.

The following table shows how these components typically map for a **User** resource:

Action	HTTP Method	Endpoint	Description
Create	POST	<code>/users</code>	Submits new user data to the server.
Read (All)	GET	<code>/users</code>	Returns a list of all users.
Read (Single)	GET	<code>/users/:id</code>	Retrieves a specific user based on their ID.
Update	PUT	<code>/users/:id</code>	Updates an existing user's data.
Delete	DELETE	<code>/users/:id</code>	Removes a user resource by ID.

Route Behavior Explained

- **POST /users**
 - Accepts user data in the request body (typically JSON) and creates a new record in the database. It often includes validation logic before persisting the data.
- **GET /users**
 - Returns a collection of users—this may support pagination or filtering through query parameters.

- **GET /users/:id**

Retrieves a specific user. The **:id** in the path is a **route parameter**, dynamically matched and passed to the handler.

- **PUT /users/:id**

Accepts updated user data for the given ID. Full replacement of the resource is expected, unlike **PATCH**, which is used for partial updates.

- **DELETE /users/:id**

Removes the user identified by the **id** parameter. The server usually returns a status code like **204 No Content** if successful.

This structure forms the backbone of a **RESTful API**, where each URL represents a resource, and HTTP methods define the action to take. By adhering to these conventions, your backend becomes intuitive to navigate, easier to document, and compatible with front-end frameworks, REST clients, and external integrations.

Creating GET, POST, PUT, DELETE Routes

The first step in building a web application is defining how it handles various types of HTTP requests. These requests correspond to the CRUD (Create, Read, Update, Delete) operations commonly used in web development:

Action	HTTP Method	Endpoint	Description
Create	POST	POST /users	Creates a new user resource.
Read (All)	GET	GET /users	Retrieves a list of all users.
Read (Single)	GET	GET /users/:id	Retrieves a specific user by ID.
Update	PUT	PUT /users/:id	Updates an existing user resource.
Delete	DELETE	DELETE /users/:id	Deletes a specific user resource.

Example of Basic CRUD with TypeScript & ESM

```
import express, { Request, Response } from 'express';

const app = express();

// Middleware to parse JSON bodies
app.use(express.json());

// In-memory "database" for demonstration
let items: { id: number; name: string }[] = [
  { id: 1, name: 'Item 1' },
  { id: 2, name: 'Item 2' }
];
```

```

// GET route to retrieve all items
app.get('/items', (req: Request, res: Response) => {
  res.json(items);
});

// GET route to retrieve a single item by ID
app.get('/items/:id', (req: Request, res: Response) => {
  const itemId = parseInt(req.params.id);
  const item = items.find(i => i.id === itemId);
  if (!item) return res.status(404).json({ message: 'Item not found' });
  res.json(item);
});

// POST route to create a new item
app.post('/items', (req: Request, res: Response) => {
  const newItem = req.body as { name: string };
  if (!newItem.name) return res.status(400).json({ message: 'Name is required' });

  newItem.id = items.length + 1;
  items.push(newItem);
  res.status(201).json(newItem); // Respond with the created item
});

// PUT route to update an existing item
app.put('/items/:id', (req: Request, res: Response) => {
  const itemId = parseInt(req.params.id);
  const updatedItem = req.body as { name?: string };
  const index = items.findIndex(i => i.id === itemId);

  if (index === -1) return res.status(404).json({ message: 'Item not found' });

  items[index] = { ...items[index], ...updatedItem }; // Merge old and new data
  res.json(items[index]);
});

// DELETE route to remove an item
app.delete('/items/:id', (req: Request, res: Response) => {
  const itemId = parseInt(req.params.id);
  const index = items.findIndex(i => i.id === itemId);

  if (index === -1) return res.status(404).json({ message: 'Item not found' });

  items.splice(index, 1); // Remove the item
  res.status(204).send(); // No content response
});

// Start the server
const PORT = 3000;
app.listen(PORT, () => {
  console.log(`Server is running on http://localhost:${PORT}`);
});

```

```
});
```

Explanation

1. **GET /items**: Returns all items in the array.
2. **GET /items/:id**: Retrieves a specific item by its ID. The **:id** is a route parameter.
3. **POST /items**: Adds a new item to the array. The request body contains the item details.
4. **PUT /items/:id**: Updates an existing item. The request body contains the updated data.
5. **DELETE /items/:id**: Removes an item from the array.

Using Express Router for Modular API Design

As web applications scale, defining all routes in a single file can quickly lead to cluttered, hard-to-navigate code. To address this, Express provides the **Router** module—a powerful feature that allows you to encapsulate and organize routes into separate, logical components.

An **Express.Router()** instance behaves like a mini-application. It supports middleware, route handlers, and HTTP method definitions just like the main **app** object. By grouping related routes (e.g., user routes, product routes, auth routes) into their own modules, you improve both readability and maintainability.

This modular approach supports:

- **Separation of concerns**: Each route module handles one resource or domain.
- **Code reuse**: Middleware or controllers can be applied at the module level.
- **Cleaner main application file**: **app.js** or **index.js** only needs to mount routers.

Example Workflow

1. Define routes in a separate module:

```
// routes/users.js
const express = require('express');
const router = express.Router();

router.get('/', (req, res) => {
  res.send('List of users');
});

router.post('/', (req, res) => {
  res.send('Create a new user');
});

module.exports = router;
```

2. Use the router in the main application:

```
// app.js
const express = require('express');
const app = express();
const userRoutes = require('./routes/users');

app.use('/users', userRoutes);

app.listen(3000, () => {
  console.log('Server running on port 3000');
});
```

In this setup, a **GET** request to **/users** will be handled by the route defined in **routes/users.js**. The prefix (**/users**) is automatically prepended to all routes in that module.

This approach not only keeps your application modular but also scales well for large projects where you may have dozens of route groups and sub-resources.

Example of Basic CRUD with Express Router

To demonstrate the benefits of modular routing in Express, let's implement a simple **CRUD API** using the **express.Router()** object in TypeScript. This example defines routes to manage a list of items, encapsulated in a dedicated module. This approach keeps your application organized and separates route logic by domain ideal for building scalable, maintainable APIs.

Step 1: Create a Router Module (**itemsRouter.ts**)

The following code defines a fully functional router for managing **items**. It supports all basic CRUD operations:

```
import express, { Request, Response } from 'express';

const router = express.Router();

// In-memory "database" for demonstration
let items: { id: number; name: string }[] = [
  { id: 1, name: 'Item 1' },
  { id: 2, name: 'Item 2' }
];

// GET route to retrieve all items
router.get('/', (req: Request, res: Response) => {
  res.json(items);
});

// GET route to retrieve a single item by ID
router.get('/:id', (req: Request, res: Response) => {
  const itemId = parseInt(req.params.id);
  const item = items.find(i => i.id === itemId);
  if (!item) return res.status(404).json({ message: 'Item not found' });
  res.json(item);
});
```

```

});

// POST route to create a new item
router.post('/', (req: Request, res: Response) => {
  const newItem = req.body as { name: string };
  if (!newItem.name) return res.status(400).json({ message: 'Name is required' });

  newItem.id = items.length + 1;
  items.push(newItem);
  res.status(201).json(newItem); // Respond with the created item
});

// PUT route to update an existing item
router.put('/:id', (req: Request, res: Response) => {
  const itemId = parseInt(req.params.id);
  const updatedItem = req.body as { name?: string };
  const index = items.findIndex(i => i.id === itemId);

  if (index === -1) return res.status(404).json({ message: 'Item not found' });

  items[index] = { ...items[index], ...updatedItem }; // Merge old and new data
  res.json(items[index]);
});

// DELETE route to remove an item
router.delete('/:id', (req: Request, res: Response) => {
  const itemId = parseInt(req.params.id);
  const index = items.findIndex(i => i.id === itemId);

  if (index === -1) return res.status(404).json({ message: 'Item not found' });

  items.splice(index, 1); // Remove the item
  res.status(204).send(); // No content response
});

export default router;

```

Key Highlights

- **Modularity:** The router is self-contained and easily imported into `app.ts` or `main.ts`.
- **CRUD Support:** Each route performs one of the Create, Read, Update, or Delete operations.
- **Error Handling:** Basic validation and `404` responses are implemented for robustness.
- **Stateless Storage:** The example uses an in-memory array, which resets when the server restarts.

Workflow Summary: CRUD Operations with the Item Router

The following endpoints represent the full set of CRUD operations supported by the `itemsRouter` module. Each route is responsible for managing a specific action on the in-memory `items` resource.

1. **GET /** – Retrieve All Items

Returns the complete list of items in JSON format. Useful for displaying all stored resources.

2. **GET /:id** – Retrieve Item by ID

Fetches a single item based on its unique `id`. If the item does not exist, the server responds with a `404 Not Found` error.

3. **POST /** – Create a New Item

Adds a new item to the list. The client sends a JSON object (e.g., `{ "name": "New Item" }`) in the request body. The server assigns a new `id` and returns the created item.

4. **PUT /:id** – Update an Existing Item

Modifies an existing item identified by `id`. The client sends updated data (e.g., `{ "name": "Updated Name" }`) in the request body. If the item exists, it is updated and returned; otherwise, a `404` response is issued.

5. **DELETE /:id** – Delete an Item

Removes the item with the specified `id` from the list. If successful, the server responds with a `204 No Content` status. If the item is not found, a `404` is returned. This workflow ensures that your router module fully supports the standard operations needed for managing a basic resource. It provides a clear foundation for building out more advanced logic, connecting to databases, or applying middleware for validation and authentication.

Step 2: Use the Router in the Main Application (`app.ts`)

In your main application file (`app.ts` or `index.ts`), you can mount the router like this:

```
import express from 'express';
import itemsRouter from './itemsRouter';

const app = express();

// Middleware to parse JSON bodies
app.use(express.json());

// Mount the router at /items
app.use('/items', itemsRouter);

// Start the server
const PORT = 3000;
app.listen(PORT, () => {
  console.log(`Server is running on http://localhost:${PORT}`);
});
```

Explanation

1. Modularization:

- The `itemsRouter.ts` file encapsulates all routes related to the `/items` endpoint.
- This keeps the main application file (`app.ts`) clean and focused on high-level concerns like middleware and server setup.

2. Router Usage:

- The `router` object from `Express.Router` is used to define routes just like you would with `app`.
- These routes are then mounted at the `/items` path in the main application using `app.use('/items', itemsRouter)`.

3. Code Organization:

- By separating the routes into their own file, you can easily reuse or extend them in the future.
- For example, if you want to add more endpoints (e.g., `/users`, `/orders`), you can create additional router files and mount them similarly.

4. Scalability:

- As your application grows, this modular approach ensures that your code remains organized and maintainable.
- You can group related routes together (e.g., all user-related routes in `usersRouter.ts`, all order-related routes in `ordersRouter.ts`).

Testing the API

Once your server is running, you can verify the functionality of the `itemsRouter` by sending requests to its endpoints using tools like **Postman**, **cURL**, or any REST client of your choice. Below is a list of testable routes, along with example request formats and expected responses.

1. GET `/items` – Retrieve All Items

Description: Fetches all items currently stored in the in-memory list.

Example Response:

```
[
  { "id": 1, "name": "Item 1" },
  { "id": 2, "name": "Item 2" }
]
```

2. GET `/items/:id` – Retrieve a Single Item

Description: Fetches a specific item by its unique ID (e.g., `/items/1`).

Example Response:

```
{ "id": 1, "name": "Item 1" }
```

3. POST `/items` – Create a New Item

Description: Adds a new item. The request must include a JSON body with a `name` field.

Request Body:

```
{ "name": "Item 3" }
```

Example Response:

```
{ "id": 3, "name": "Item 3" }
```

4. PUT `/items/:id` – Update an Existing Item

Description: Updates the name of an existing item identified by its ID.

Request Body:

```
{ "name": "Updated Item 1" }
```

Example Response:

```
{ "id": 1, "name": "Updated Item 1" }
```

5. DELETE `/items/:id` – Delete an Item

Description: Deletes an item from the list based on its ID.

Response: Returns HTTP status code `204 No Content` if deletion is successful (no body).

These test cases confirm that your API supports all core CRUD operations and handles both data manipulation and response formatting correctly.

Implementing Middleware for Request Processing

Middleware is a foundational concept in Express.js that allows developers to **intercept and process HTTP requests** at various stages of their lifecycle. Middleware functions act as intermediaries—executed in sequence—between the incoming request and the final route handler or response.

Middleware can be used to:

- Log requests
- Authenticate users
- Validate input
- Modify request or response objects
- Handle errors globally

Understanding Middleware Structure

A middleware function in Express receives **three arguments**:

- **req** – The request object, containing details like headers, query parameters, and body data.
- **res** – The response object, used to construct and send responses.
- **next** – A callback that passes control to the next middleware function in the stack.

To function properly, middleware must either **terminate the request-response cycle** (e.g., by sending a response) or call `next()` to pass control to the next handler.

Creating Custom Middleware

You can define your own middleware to perform specific tasks. Below is an example of a simple logging middleware that prints the timestamp, HTTP method, and requested URL:

```
import express, { Request, Response, NextFunction } from 'express';

const app = express();

// Logging middleware
const logger = (req: Request, res: Response, next: NextFunction): void => {
  const timestamp = new Date().toISOString();
  console.log(`${timestamp} - ${req.method} ${req.url}`);
  next(); // Pass control to the next middleware
};

// Apply the middleware globally
app.use(logger);

// Start the server
const PORT = 3000;
app.listen(PORT, () => {
  console.log(`Server is running on http://localhost:${PORT}`);
});
```

Explanation

- **Logs request metadata** to the console for monitoring or debugging.
- **Invokes `next()`** so that the request can continue through the middleware chain or reach the intended route handler.

Middleware can be applied:

- Globally using `app.use()` (as in the example above)
- To specific routes only
- As error-handling functions (using four parameters: `err`, `req`, `res`, `next`)

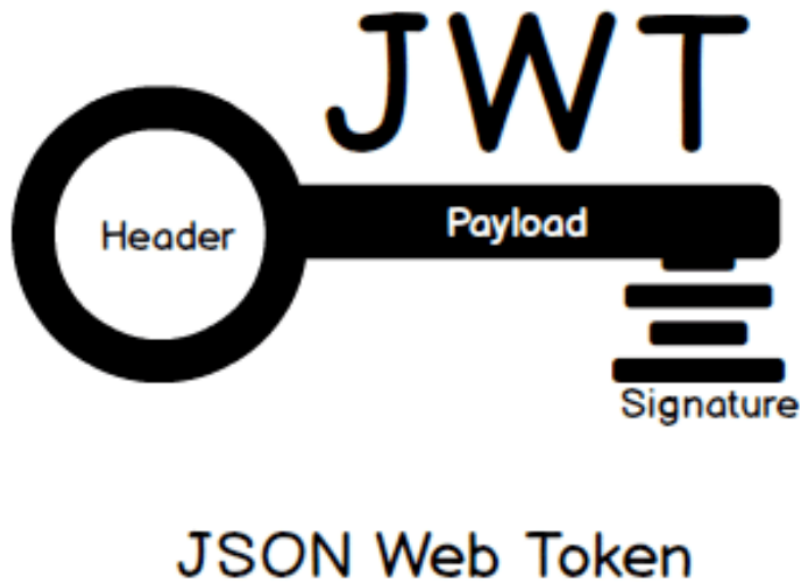
This modular approach makes Express.js applications highly extensible and maintainable, especially as complexity grows.

Introduction to Setting Up Authentication Routes

Authentication is a critical part of any secure web application. It verifies the identity of users and ensures that only authorized individuals can access protected resources. Without proper authentication, sensitive operations—such as viewing personal data, modifying records, or accessing administrative features—can be exposed to unauthorized access. In modern applications, authentication is typically implemented using **token-based systems** such as **JSON Web Tokens (JWT)**, and passwords are securely managed using hashing libraries like **bcrypt**.

JWT (JSON Web Token)

JWT, or **JSON Web Token**, is an open standard (RFC 7519) that defines a compact, URL-safe way of representing claims between two parties. It is widely used in web development for securely transmitting information, particularly for **authentication and authorization**. A JWT is a self-contained token that can be passed between client and server, allowing the server to verify the user's identity without storing session data.



Structure of a JWT

A typical JWT consists of **three parts**, each encoded in Base64 and separated by dots:

```
xxxxx.yyyyy.zzzzz
```

1. Header

The header specifies metadata about the token, including the signing algorithm being used (e.g., **HS256** for HMAC SHA-256).

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

2. Payload

The payload contains **claims**—statements about the user or token subject. Common claims include the user ID (**sub**), roles, or token expiration (**exp**).

```
{
  "sub": "1234567890",
  "name": "John Doe",
  "admin": true
}
```

3. Signature

The signature is created by hashing the encoded header and payload with a secret key. This ensures that the token has not been tampered with.

```
HMACHA256(  
  base64UrlEncode(header) + "." + base64UrlEncode(payload),  
  secret  
)
```

Why JWTs Are Used

JWTs are popular in modern web applications for several reasons:

- **Stateless Authentication:** No need to store session data on the server; all required information is encoded in the token.
- **Compact & Efficient:** JWTs are small in size and can be included in HTTP headers, URLs, or cookies.
- **Integrity & Security:** The signature ensures that the payload hasn't been altered and verifies the sender's identity.
- **Cross-domain Use:** JWTs work well in distributed systems and APIs, including Single Sign-On (SSO) setups.

In practice, JWTs are issued by the server upon successful login, sent to the client, and then attached to future requests (usually in the **Authorization** header as **Bearer <token>**). On each request, the server verifies the token before granting access to protected resources.

Setting Up JWT-Based Authentication

To implement secure, token-based user authentication, we'll use **JSON Web Tokens (JWT)** to issue signed tokens upon successful login, and **bcrypt** to hash and compare passwords securely. This setup enables a stateless authentication mechanism where the server doesn't need to maintain sessions—each client request includes a signed token that proves the user's identity.

Step 1: Install Required Packages

Use the following command to install the necessary libraries

```
npm install jsonwebtoken bcryptjs express
```

Installed Packages

- **jsonwebtoken**

Used to create and verify signed tokens. JWTs encapsulate user identity and permissions and are sent with every client request to protected endpoints.

- **bcryptjs**

Provides hashing and comparison functions for securely handling passwords. Passwords are never stored in plain text.

- **express**

A minimal and flexible Node.js web framework used to build the API endpoints.

Step 2: Configure JWT Authentication

In this step, we configure JWT-based authentication by implementing two core functions: one to **generate tokens** and another to **verify them** on protected routes. We'll also define a simulated user database and middleware to parse and validate incoming JSON payloads.

JWT Setup Code


```

import express, { Request, Response, NextFunction } from 'express';
import jwt from 'jsonwebtoken';
import bcrypt from 'bcryptjs';

const app = express();

// Middleware to parse JSON bodies
app.use(express.json());

// Simulated user database
let users: { id: number; username: string; password: string }[] = [];

// Secret key for signing JWTs
const SECRET_KEY = 'your-secret-key';

// Helper function to generate JWT
function generateToken(user: { id: number }): string {
  return jwt.sign({ userId: user.id }, SECRET_KEY, { expiresIn: '1h' });
}

// Middleware to verify JWT
function authenticateToken(req: Request, res: Response, next:
NextFunction): void {
  const authHeader = req.headers['authorization'];
  const token = authHeader && authHeader.split(' ')[1]; // Extract token
  from "Bearer <token>"
  if (!token) return res.status(401).json({ message: 'Access denied. No
token provided.' });

  jwt.verify(token, SECRET_KEY, (err, decoded) => {
    if (err) return res.status(403).json({ message: 'Invalid or expired
token.' });
    req.user = decoded as { userId: number }; // Attach user information to
the request object
    next();
  });
}

```

Key Components Explained

- **generateToken(user)**

Creates a signed JWT that contains the user's unique ID (**userId**). The token is valid for 1 hour. This is typically issued upon successful login.

- **authenticateToken(req, res, next)**

- Acts as middleware to protect routes. It:
- Extracts the token from the **Authorization** header
- Verifies the token's signature and expiration
- Attaches the decoded payload to **req.user** if valid

- Rejects access with appropriate status codes if the token is missing or invalid

Simulated User Database

For this demonstration, we're using an in-memory `users` array to store credentials. In a real-world application, this would be replaced with a persistent store like PostgreSQL, MongoDB, or Firebase.

Secret Key Considerations

- The `SECRET_KEY` is used for both signing and verifying JWTs.
 - In production, it should be stored securely (e.g., in environment variables) and kept private to the server.
-

Step 3: Create Authentication Routes

Now that we've configured JWT generation and verification, it's time to implement the actual authentication endpoints: one for registering users, one for logging them in, and one protected route to demonstrate JWT enforcement. These routes use `bcryptjs` to securely hash and compare passwords, and `jsonwebtoken` to issue a signed access token upon successful login.

Code Implementation

```

// Register Route
app.post('/register', async (req: Request, res: Response) => {
  const { username, password } = req.body as { username: string; password: string };

  // Check if the user already exists
  const existingUser = users.find(u => u.username === username);
  if (existingUser) return res.status(400).json({ message: 'Username already exists.' });

  // Hash the password
  const hashedPassword = await bcrypt.hash(password, 10);

  // Create a new user
  const newUser = { id: users.length + 1, username, password: hashedPassword };
  users.push(newUser);

  res.status(201).json({ message: 'User registered successfully!' });
});

// Login Route
app.post('/login', async (req: Request, res: Response) => {
  const { username, password } = req.body as { username: string; password: string };

  // Find the user by username
  const user = users.find(u => u.username === username);
  if (!user) return res.status(401).json({ message: 'Invalid credentials.' });

  // Verify the password
  const isValidPassword = await bcrypt.compare(password, user.password);
  if (!isValidPassword) return res.status(401).json({ message: 'Invalid credentials.' });

  // Generate a JWT
  const token = generateToken(user);
  res.json({ token });
});

// Protected Route
app.get('/dashboard', authenticateToken, (req: Request, res: Response) => {
  res.json({ message: `Welcome to your dashboard, User ${req.user.userId}` });
});

```

Route Breakdown

1. **POST /register**

- Registers a new user.

- Ensures username uniqueness.
- Hashes the password using `bcrypt` before storage.
- Responds with a success message after adding the user to the simulated database.

2. `POST /login`

- Authenticates the user by comparing the entered password with the hashed password in storage.
- If successful, returns a signed JWT to the client.
- This token will be used to access protected routes.

3. `GET /dashboard`

- A secured route that requires a valid JWT.
- Uses `authenticateToken` middleware to verify the request.
- Returns a personalized message containing the authenticated user's ID.

Middleware Summary

- `express.json()`

Parses incoming JSON request bodies so values like `username` and `password` are available on `req.body`.

- `authenticateToken(req, res, next)`

Middleware function that:

- Extracts and verifies the JWT from the `Authorization` header.
- Rejects unauthorized access.
- Attaches decoded user info (`userId`) to the request object if valid.

This completes a basic yet secure JWT authentication flow with route protection. You now have a working setup for user registration, login, and token-based access control.

Step 4: Logout Route

In a JWT-based authentication system, logout functionality differs from traditional session-based approaches. Since JWTs are **stateless and self-contained**, the server does not maintain session state or store token metadata by default. As a result, logging out simply means that the **client must discard the token**.

Logout Route Implementation

```
// Simulated logout endpoint (client-controlled)
app.post('/logout', (req: Request, res: Response) => {
  res.json({ message: 'Logged out successfully. Please discard the token.'
});
});
```

Route Behavior

- **POST /logout**

This route does not invalidate the token server-side but informs the client that they should delete the stored token (from localStorage, cookies, etc.). In more advanced setups, the server may maintain a **token blacklist** or **revocation list** to explicitly block known tokens before their expiration.

How It All Works Together

Here's a quick recap of the full JWT authentication flow:

1. **Register (POST /register)**

A new user is created with a hashed password and stored in the in-memory user database.

2. **Login (POST /login)**

A valid username/password pair results in a signed JWT being returned to the client.

3. **Protected Route (GET /dashboard)**

Middleware (**authenticateToken**) verifies the JWT before allowing access to protected resources.

4. **Logout (POST /logout)**

The client discards the token, effectively ending the session from their end.

This completes a fully functional, token-based authentication system using Express, **jsonwebtoken**, and **bcryptjs**. It's suitable for building scalable, stateless APIs where clients are responsible for token handling and renewal.

Conclusion

This lesson has equipped you with the foundational knowledge and practical skills needed to handle routes and middleware effectively in Express.js applications. You've learned how to create essential CRUD operations, organize your routes for scalability using Express Router, and implement middleware for enhanced request processing. Additionally, you've explored setting up secure authentication with JWTs, ensuring your application is both functional and secure. With these skills, you are now prepared to design robust APIs capable of handling complex workflows and growing user demands.